

Header And Logo



Peripheral Links

[Donate to FreeBSD](#)

Search

Search

Site Navigation

- [Home](#)
- [About](#)
 - [Introduction](#)
 - [Features](#)
 - [Advocacy](#)
 - [Marketing](#)
- [Get FreeBSD](#)
 - [Release Information](#)
 - [Release Engineering](#)
- [Documentation](#)
 - [FAQ](#)
 - [Handbook](#)
 - [Porter's Handbook](#)
 - [Developer's Handbook](#)
 - [Manual Pages](#)
 - [Documentation Project Primer](#)
 - [All Books and Articles](#)
- [Community](#)
 - [Mailing Lists](#)
 - [Forums](#)
 - [User Groups](#)
 - [Events](#)
- [Developers](#)
 - [Project Ideas](#)
 - [GIT Repository](#)
- [Support](#)
 - [Vendors](#)
 - [Security Information](#)
 - [Bug Reports](#)
 - [Submit Bug-report](#)
- [Foundation](#)
 - [Donate](#)

FreeBSD Manual Pages

geli

man

apropos

All Sections

FreeBSD 15.1-RELEASE and Ports.quarterly

html

[home](#) | [help](#)

NAME

geli -- control utility for the cryptographic GEOM class

SYNOPSIS

To compile GEOM_ELI into your kernel, add the following lines to your kernel configuration file:

```
device crypto
options GEOM_ELI
```

Alternatively, to load the GEOM_ELI module at boot time, add the following line to your [loader.conf\(5\)](#):

```
geom_eli_load="YES"
```

Usage of the **geli** utility:

```
geli init [-bdgPRTv] [-a aalgo] [-B backupfile] [-e ealgo] [-i iterations]
          [-J newpassfile] [-K newkeyfile] [-l keylen] [-s sectorsize]
          [-V version] prov ...
geli label - an alias for init
geli attach [-Cdprv] [-n keyno] [-j passfile] [-k keyfile] prov ...
geli detach [-fl] prov ...
geli stop - an alias for detach
geli onetime [-dRT] [-a aalgo] [-e ealgo] [-l keylen] [-s sectorsize] prov
geli configure [-bBdDgGrRtT] prov ...
geli setkey [-pPv] [-i iterations] [-j passfile] [-J newpassfile]
          [-k keyfile] [-K newkeyfile] [-n keyno] prov
geli delkey [-afv] [-n keyno] prov
geli kill [-av] [prov ...]
geli backup [-v] prov file
geli restore [-fv] file prov
geli suspend [-v] -a | prov ...
geli resume [-pv] [-j passfile] [-k keyfile] prov
geli resize [-v] -s oldsize prov
geli version [prov ...]
geli clear [-v] prov ...
geli dump [-v] prov ...
geli list
geli status
geli load
geli unload
```

DESCRIPTION

The **geli** utility is used to configure encryption on GEOM providers.

The following is a list of the most important features:

- * Utilizes the [crypto\(9\)](#) framework, so when there is crypto hardware available, **geli** will make use of it automatically.
- * Supports many cryptographic algorithms (currently **AES-XTS**, **AES-CBC**, and **Camellia-CBC**).
- * Can optionally perform data authentication (integrity verification) utilizing one of the following algorithms: **HMAC/SHA1**, **HMAC/RIPEMD160**, **HMAC/SHA256**, **HMAC/SHA384** or **HMAC/SHA512**.
- * Can create a User Key from up to two, piecewise components: a passphrase entered via prompt or read from one or more passfiles; a keyfile read from one or more files.

- * Allows encryption of the root partition. The user is asked for the passphrase before the root filesystem is mounted.
- * Strengthens the passphrase component of the User Key with: B. Kaliski, *PKCS #5: Password-Based Cryptography Specification, Version 2.0.*, RFC, 2898.
- * Allows the use of two independent User Keys (e.g., a "user key" and a "company key").
- * It is fast - **geli** performs simple sector-to-sector encryption.
- * Allows the encrypted Master Key to be backed up and restored, so that if a user has to quickly destroy key material, it is possible to get the data back by restoring keys from backup.
- * Providers can be configured to automatically detach on last close, so users do not have to remember to detach providers after unmounting the filesystems.
- * Allows attaching a provider with a random, one-time Master Key, which is useful for swap partitions and temporary filesystems.
- * Allows verification of data integrity (data authentication).
- * Allows suspending and resuming encrypted devices.

The first argument to **geli** indicates an action to be performed:

init Initialize providers which need to be encrypted. If multiple providers are listed as arguments, they will all be initialized with the same passphrase and/or User Key. A unique salt will be randomly generated for each provider to ensure the Master Key for each is unique. Here you can set up the cryptographic algorithm to use, Data Key length, etc. The last sector of the providers is used to store metadata. The **init** subcommand also automatically writes metadata backups to `/var/backups/<prov>.eli` file. The metadata can be recovered with the **restore** subcommand described below.

Additional options include:

- a *aalgo* Enable data integrity verification (authentication) using the given algorithm. This will reduce the size of storage available and also reduce speed. For example, when using 4096 bytes sector and **HMAC/SHA256** algorithm, 89% of the original provider storage will be available for use. Currently supported algorithms are: **HMAC/SHA1**, **HMAC/RIPEMD160**, **HMAC/SHA256**, **HMAC/SHA384** and **HMAC/SHA512**. If the option is not given, there will be no authentication, only encryption. The recommended algorithm is **HMAC/SHA256**.
- b Try to decrypt this partition during boot, before the root partition is mounted. This makes it possible to use an encrypted root partition. One will still need bootable unencrypted storage with a `/boot/` directory, which can be a CD-ROM disc or USB pen-drive, that can be removed after boot.
- B *backupfile* File name to use for metadata backup instead of the default `/var/backups/<prov>.eli`. To inhibit backups, you can use *none* as the *backupfile*. If multiple providers were initialized in the one command, you can use *PROV* (all upper-case) in the file name, and it will be replaced with the provider name. If *PROV* is not found in the file name and multiple providers were initialized in the one command, `-<prov>` will be appended to the end of the file name specified.
- d When entering the passphrase to boot from this encrypted root filesystem, echo ``*'` characters. This makes the length of the passphrase visible.
- e *ealgo* Encryption algorithm to use. Currently supported algorithms are: **AES-XTS**, **AES-CBC**, **Camellia-CBC**, and **NULL**. The default and recommended algorithm is **AES-XTS**. **NULL** is unen-

encrypted.

- g Enable booting from this encrypted root filesystem. The boot loader prompts for the passphrase and loads *loader(8)* from the encrypted partition.
- i *iterations* Number of iterations to use with PKCS#5v2 when processing User Key passphrase component. If this option is not specified, *geli* will find the number of iterations which is equal to 2 seconds of crypto work. If 0 is given, PKCS#5v2 will not be used. PKCS#5v2 processing is performed once, after all parts of the passphrase component have been read.
- J *newpassfile* Specifies a file which contains the passphrase component of the User Key (or part of it). If *newpassfile* is given as -, standard input will be used. Only the first line (excluding newline character) is taken from the given file. This argument can be specified multiple times, which has the effect of reassembling a single passphrase split across multiple files. Cannot be combined with the -P option.
- K *newkeyfile* Specifies a file which contains the keyfile component of the User Key (or part of it). If *newkeyfile* is given as -, standard input will be used. This argument can be specified multiple times, which has the effect of reassembling a single keyfile split across multiple keyfile parts.
- l *keylen* Data Key length to use with the given cryptographic algorithm. If the length is not specified, the selected algorithm uses its *default* key length.

AES-XTS
128, 256

AES-CBC, Camellia-CBC
128, 192, 256

- P Do not use a passphrase as a component of the User Key. Cannot be combined with the -J option.
- s *sectorsize* Change decrypted provider's sector size. Increasing the sector size allows increased performance, because encryption/decryption which requires an initialization vector is done per sector; fewer sectors means less computational work.
- R Turn off automatic expansion. By default, if the underlying provider grows, the encrypted provider will grow automatically too. The metadata will be moved to the new location. If automatic expansion is turned off and the underlying provider changes size, attaching encrypted provider will no longer be possible as the metadata will no longer be located in the last sector. In this case *GELI* will only log the previous size of the underlying provider, so metadata can be found easier, if resize was done by mistake.
- T Don't pass through BIO_DELETE calls (i.e., TRIM/UNMAP). This can prevent an attacker from knowing how much space you're actually using and which sectors contain live data, but will also prevent the backing store (SSD, etc) from re-

claiming space you're not using, which may degrade its performance and lifespan. The underlying provider may or may not actually obliterate the deleted sectors when TRIM is enabled, so it should not be considered to add any security.

-V version Metadata version to use. This option is helpful when creating a provider that may be used by older **FreeBSD/GELI** versions. Consult the "HISTORY" section to find which metadata version is supported by which FreeBSD version. Note that using an older version of metadata may limit the number of features available.

attach Attach the given providers. The encrypted Master Keys are loaded from the metadata and decrypted using the given passphrase/keyfile and new GEOM providers are created using the specified provider names. A ".eli" suffix is added to the user specified provider names. Multiple providers can only be attached with a single **attach** command if they all have the same passphrase and keyfiles.

Additional options include:

-C Do a dry-run decryption. This is useful to verify passphrase and keyfile without decrypting the device.

-d If specified, the decrypted providers are detached automatically on last close, so the user does not have to remember to detach providers after unmounting the filesystems. This only works when providers were opened for writing, and will not work if the filesystems on the providers were mounted read-only. Probably a better choice is the **-l** option for the **detach** subcommand.

-n keyno Specifies the index number of the Master Key copy to use (could be 0 or 1). If the index number is not provided all keys will be tested.

-j passfile Specifies a file which contains the passphrase component of the User Key (or part of it). For more information see the description of the **-J** option for the **init** subcommand. The same passfiles are used for all listed providers.

-k keyfile Specifies a file which contains the keyfile component of the User Key (or part of it). For more information see the description of the **-K** option for the **init** subcommand. The same keyfiles are used for all listed providers.

-p Do not use a passphrase as a component of the User Keys. Cannot be combined with the **-j** option.

-r Attach read-only providers. They are not opened for writing.

detach Detach the given providers, which means remove the devfs entry and clear the Master Key and Data Keys from memory.

Additional options include:

-f Force detach - detach even if the provider is open.

-l Mark provider to detach on last close, after the last filesystem has been unmounted. If this option is specified, the provider will not be detached while it is open, but will be automatically detached when it is closed for the last time even if it was only opened for reading.

onetime Attach the given providers with a random, one-time (ephemeral) Master Key. The command can be used to encrypt swap partitions

or temporary filesystems.

Additional options include:

- a *aalgo* Enable data integrity verification (authentication). For more information, see the description of the **init** subcommand.
- e *ealgo* Encryption algorithm to use. For more information, see the description of the **init** subcommand.
- d Detach on last close, after the last filesystem has been unmounted. Note: this option is not usable for temporary filesystems as the provider is detached after the filesystem has been created. It still can, and should, be used for swap partitions. For more information, see the description of the **attach** subcommand.
- l *keylen* Data Key length to use with the given cryptographic algorithm. For more information, see the description of the **init** subcommand.
- s *sectorsize* Change decrypted provider's sector size. For more information, see the description of the **init** subcommand.
- R Turn off automatic expansion. For more information, see the description of the **init** subcommand.
- T Disable TRIM/UNMAP passthru. For more information, see the description of the **init** subcommand.

configure Change configuration of the given providers.

Additional options include:

- b Set the BOOT flag on the given providers. For more information, see the description of the **init** subcommand.
- B Remove the BOOT flag from the given providers.
- d When entering the passphrase to boot from this encrypted root filesystem, echo '*' characters. This makes the length of the passphrase visible.
- D Disable echoing of any characters when a passphrase is entered to boot from this encrypted root filesystem. This hides the passphrase length.
- g Enable booting from this encrypted root filesystem. The boot loader prompts for the passphrase and loads [loader\(8\)](#) from the encrypted partition.
- G Deactivate booting from this encrypted root partition.
- r Turn on automatic expansion. For more information, see the description of the **init** subcommand.
- R Turn off automatic expansion.
- t Enable TRIM/UNMAP passthru. For more information, see the description of the **init** subcommand.
- T Disable TRIM/UNMAP passthru.

setkey Install a copy of the Master Key into the selected slot, encrypted with a new User Key. If the selected slot is populated, replace the existing copy. A provider has one Master Key, which can be stored in one or both slots, each encrypted with an independent User Key. With the **init** subcommand, only key number 0 is initialized. The User Key can be changed at any time: for an attached provider, for a detached provider, or on the backup file. When a provider is attached, the user does not have to

provide an existing passphrase/keyfile.

Additional options include:

- i *iterations* Number of iterations to use with PKCS#5v2. If 0 is given, PKCS#5v2 will not be used. To be able to use this option with the **setkey** subcommand, only one key has to be defined and this key must be changed.
- j *passfile* Specifies a file which contains the passphrase component of a current User Key (or part of it).
- J *newpassfile* Specifies a file which contains the passphrase component of the new User Key (or part of it).
- k *keyfile* Specifies a file which contains the keyfile component of a current User Key (or part of it).
- K *newkeyfile* Specifies a file which contains the keyfile component of the new User Key (or part of it).
- n *keyno* Specifies the index number of the Master Key copy to change (could be 0 or 1). If the provider is attached and no key number is given, the key used for attaching the provider will be changed. If the provider is detached (or we are operating on a backup file) and no key number is given, the first Master Key copy to be successfully decrypted with the provided User Key passphrase/keyfile will be changed.
- p Do not use a passphrase as a component of the current User Key. Cannot be combined with the -j option.
- P Do not use a passphrase as a component of the new User Key. Cannot be combined with the -J option.

delkey Destroy (overwrite with random data) the selected Master Key copy. If one is destroying keys for an attached provider, the provider will not be detached even if all copies of the Master Key are destroyed. It can even be rescued with the **setkey** subcommand because the Master Key is still in memory.

Additional options include:

- a Destroy all copies of the Master Key (does not need -f option).
- f Force key destruction. This option is needed to destroy the last copy of the Master Key.
- n *keyno* Specifies the index number of the Master Key copy. If the provider is attached and no key number is given, the key used for attaching the provider will be destroyed. If provider is detached (or we are operating on a backup file) the key number has to be given.

kill This command should be used only in emergency situations. It will destroy all copies of the Master Key on a given provider and will detach it forcibly (if it is attached). This is absolutely a one-way command - if you do not have a metadata backup, your data is gone for good. In case the provider was attached with the -r flag, the keys will not be destroyed, only the provider will be detached.

Additional options include:

- a If specified, all currently attached providers will be killed.

backup Backup metadata from the given provider to the given file.

restore Restore metadata from the given file to the given provider.

Additional options include:

-f Metadata contains the size of the provider to ensure that the correct partition or slice is attached. If an attempt is made to restore metadata to a provider that has a different size, **geli** will refuse to restore the data unless the **-f** switch is used. If the partition or slice has been grown, the **resize** subcommand should be used rather than attempting to relocate the metadata through **backup** and **restore**.

suspend Suspend device by waiting for all inflight requests to finish, clearing all sensitive information such as the Master Key and Data Keys from kernel memory, and blocking all further I/O requests until the **resume** subcommand is executed. This functionality is useful for laptops. Suspending a laptop should not leave an encrypted device attached. The **suspend** subcommand can be used rather than closing all files and directories from filesystems on the encrypted device, unmounting the filesystem, and detaching the device. Any access to the encrypted device will be blocked until the Master Key is reloaded through the **resume** subcommand. Thus there is no need to close nor unmount anything. The **suspend** subcommand does not work with devices created with the **onetime** subcommand. Please note that sensitive data might still be present in memory locations such as the filesystem cache after suspending an encrypted device.

Additional options include:

-a Suspend all **geli** devices.

resume Resume previously suspended device. The caller must ensure that executing this subcommand does not access the suspended device, leading to a deadlock. For example, suspending a device which contains the filesystem where the **geli** utility is stored is a bad idea.

Additional options include:

-j passfile Specifies a file which contains the passphrase component of the User Key, or part of it. For more information see the description of the **-J** option for the **init** subcommand.

-k keyfile Specifies a file which contains the keyfile component of the User Key, or part of it. For more information see the description of the **-K** option for the **init** subcommand.

-p Do not use a passphrase as a component of the User Key. Cannot be combined with the **-j** option.

resize Inform **geli** that the provider has been resized. The old metadata block is relocated to the correct position at the end of the provider and the provider size is updated.

Additional options include:

-s oldsize The size of the provider before it was resized.

version If no arguments are given, the **version** subcommand will print the version of **geli** userland utility as well as the version of the ELI GEOM class.

If GEOM providers are specified, the **version** subcommand will print metadata version used by each of them.

clear Clear metadata from the given providers. **WARNING:** This will erase with zeros the encrypted Master Key copies stored in the metadata.

dump Dump metadata stored on the given providers.

list See [geom\(8\)](#).
status See [geom\(8\)](#).
load See [geom\(8\)](#).
unload See [geom\(8\)](#).

Additional options include:

-v Be more verbose.

KEY SUMMARY

Master Key

Upon **init**, the **geli** utility generates a random Master Key for the provider. The Master Key never changes during the lifetime of the provider. Each copy of the provider metadata, active or backed up to a file, can store up to two, independently-encrypted copies of the Master Key.

User Key

Each stored copy of the Master Key is encrypted with a User Key, which is generated by the **geli** utility from a passphrase and/or a keyfile. The **geli** utility first reads all parts of the keyfile in the order specified on the command line, then reads all parts of the stored passphrase in the order specified on the command line. If no passphrase parts are specified, the system prompts the user to enter the passphrase. The passphrase is optionally strengthened by PKCS#5v2. The User Key is a digest computed over the concatenated keyfile and passphrase.

Data Key

During operation, one or more Data Keys are deterministically derived by the kernel from the Master Key and cached in memory. The number of Data Keys used by a given provider, and the way they are derived, depend on the GELI version and whether the provider is configured to use data authentication.

SYSCTL VARIABLES

The following [sysctl\(8\)](#) variables can be used to control the behavior of the **ELI** GEOM class. The default value is shown next to each variable. Some variables can also be set in */boot/loader.conf*.

kern.geom.eli.version

Version number of the **ELI** GEOM class.

kern.geom.eli.debug: 0

Debug level of the **ELI** GEOM class. This can be set to a number between 0 and 3 inclusive. If set to 0, minimal debug information is printed. If set to 3, the maximum amount of debug information is printed.

kern.geom.eli.tries: 3

Number of times a user is asked for the passphrase. This is only used for providers which are attached on boot, before the root filesystem is mounted. If set to 0, attaching providers on boot will be disabled. This variable should be set in */boot/loader.conf*.

kern.geom.eli.overwrites: 5

Specifies how many times the Master Key is overwritten with random values when it is destroyed. After this operation it is filled with zeros.

kern.geom.eli.use_uma_bytes

geli must allocate a buffer for every write operation, used when performing encryption. This **sysctl** reports the maximum size in bytes for which **geli** will perform the allocation using [uma\(9\)](#), as opposed to [malloc\(9\)](#).

kern.geom.eli.visible_passphrase: 0

If set to 1, the passphrase entered on boot will be visible. This alternative should be used with caution as the entered passphrase can be logged and exposed via [dmesg\(8\)](#). This variable should be set in */boot/loader.conf*.

kern.geom.eli.threads: 0

Specifies how many kernel threads should be used for doing software cryptography. Its purpose is to increase performance on SMP systems. If set to 0, a CPU-pinned thread will be started for every active CPU. Note that this variable must be set prior to attaching *geli* to a disk.

kern.geom.eli.batch: 0

When set to 1, can speed-up crypto operations by using batching. Batching reduces the number of interrupts by responding to a group of crypto requests with one interrupt. The crypto card and the driver have to support this feature.

kern.geom.eli.key_cache_limit: 8192

Specifies how many Data Keys to cache. The default limit (8192 keys) will allow caching of all keys for a 4 TiB provider with 512 byte sectors and will take around 1 MiB of memory.

kern.geom.eli.key_cache_hits

Reports how many times we were looking up a Data Key and it was already in cache. This sysctl is not updated for providers that need fewer Data Keys than the limit specified in *kern.geom.eli.key_cache_limit*.

kern.geom.eli.key_cache_misses

Reports how many times we were looking up a Data Key and it was not in cache. This sysctl is not updated for providers that need fewer Data Keys than the limit specified in *kern.geom.eli.key_cache_limit*.

kern.geom.eli.unmapped_io

Enable support for unmapped I/O buffers, currently implemented only on 64-bit platforms. This is an optimization which reduces the overhead of I/O processing. This variable is intended for debugging purposes and must be set in */boot/loader.conf*.

PERFORMANCE CONSIDERATIONS

The default value of *kern.geom.eli.threads* is usually good for a system with one SSD. However, it may need to be lowered on systems with many disks, so as to avoid creating too much thread-switching overhead. On systems with more disks than CPUs, it's best to set this variable to 1.

geli internally uses *malloc(9)* to allocate memory for operations larger than *kern.geom.eli.use_uma_bytes*, but *malloc* is slow for allocations larger than *vm.kmem_zmax*. So it's best to avoid writing more than *MAX(kern.geom.eli.use_uma_bytes, vm.kmem_zmax)* in a single write operation. On systems that format *zfs(4)* on top of *geli*, the maximum write size can be controlled by *vfs.zfs.vdev.aggregation_limit* and *vfs.zfs.vdev.aggregation_limit_non_rotating* for HDDs and SSDs, respectively.

EXIT STATUS

Exit status is 0 on success, and 1 if the command fails.

EXAMPLES

Initialize a provider which is going to be encrypted with a passphrase and random data from a file on the user's pen drive. Use 4 KiB sector size. Attach the provider, create a filesystem, and mount it. Do the work. Unmount the provider and detach it:

```
# dd if=/dev/random of=/mnt/pendrive/da2.key bs=64 count=1
# geli init -s 4096 -K /mnt/pendrive/da2.key /dev/da2
Enter new passphrase:
Reenter new passphrase:
# geli attach -k /mnt/pendrive/da2.key /dev/da2
Enter passphrase:
# dd if=/dev/random of=/dev/da2.eli bs=1m
# newfs /dev/da2.eli
# mount /dev/da2.eli /mnt/secret
...
# umount /mnt/secret
# geli detach da2.eli
```

Create an encrypted provider, but use two User Keys: one for your employee and one for you as the company's security officer (so it is not a tragedy if the employee "accidentally" forgets his passphrase):

```
# geli init /dev/da2
Enter new passphrase: (enter security officer's passphrase)
Reenter new passphrase:
# geli setkey -n 1 /dev/da2
Enter passphrase: (enter security officer's passphrase)
Enter new passphrase: (let your employee enter his passphrase ...)
Reenter new passphrase: (... twice)
```

You are the security officer in your company. Create an encrypted provider for use by the user, but remember that users forget their passphrases, so backup the Master Key with your own random key:

```
# dd if=/dev/random of=/mnt/pendrive/keys/`hostname` bs=64 count=1
# geli init -P -K /mnt/pendrive/keys/`hostname` /dev/ada0s1e
# geli backup /dev/ada0s1e /mnt/pendrive/backups/`hostname`
(use key number 0, so the encrypted Master Key will be re-encrypted by this)
# geli setkey -n 0 -k /mnt/pendrive/keys/`hostname` /dev/ada0s1e
(allow the user to enter his passphrase)
Enter new passphrase:
Reenter new passphrase:
```

Encrypted swap partition setup:

```
# dd if=/dev/random of=/dev/ada0s1b bs=1m
# geli onetime -d ada0s1b
# swapon /dev/ada0s1b.eli
```

The example below shows how to configure two providers which will be attached on boot, before the root filesystem is mounted. One of them is using passphrase and three keyfile parts and the other is using only a keyfile in one part:

```
# dd if=/dev/random of=/dev/da0 bs=1m
# dd if=/dev/random of=/boot/keys/da0.key0 bs=32k count=1
# dd if=/dev/random of=/boot/keys/da0.key1 bs=32k count=1
# dd if=/dev/random of=/boot/keys/da0.key2 bs=32k count=1
# geli init -b -K /boot/keys/da0.key0 -K /boot/keys/da0.key1 -K /boot/keys/da0.key2 da0
Enter new passphrase:
Reenter new passphrase:
# dd if=/dev/random of=/dev/da1s3a bs=1m
# dd if=/dev/random of=/boot/keys/da1s3a.key bs=128k count=1
# geli init -b -P -K /boot/keys/da1s3a.key da1s3a
```

The providers are initialized, now we have to add these lines to */boot/loader.conf*:

```
geli_da0_keyfile0_load="YES"
geli_da0_keyfile0_type="da0:geli_keyfile0"
geli_da0_keyfile0_name="/boot/keys/da0.key0"
geli_da0_keyfile1_load="YES"
geli_da0_keyfile1_type="da0:geli_keyfile1"
geli_da0_keyfile1_name="/boot/keys/da0.key1"
geli_da0_keyfile2_load="YES"
geli_da0_keyfile2_type="da0:geli_keyfile2"
geli_da0_keyfile2_name="/boot/keys/da0.key2"

geli_da1s3a_keyfile0_load="YES"
geli_da1s3a_keyfile0_type="da1s3a:geli_keyfile0"
geli_da1s3a_keyfile0_name="/boot/keys/da1s3a.key"
```

If there is only one keyfile, the index might be omitted:

```
geli_da1s3a_keyfile_load="YES"
geli_da1s3a_keyfile_type="da1s3a:geli_keyfile"
geli_da1s3a_keyfile_name="/boot/keys/da1s3a.key"
```

By convention, these loader variables are called *geli_<device>_load*. However, the actual name prefix before *_load*, *_type*, or *_name* does not matter. At boot time, the **geli** module searches through all *<prefix>_type*-like variables that have a value of "*<device>:geli_keyfile*". The paths to keyfiles

are then extracted from `<prefix>_name` variables. In the example above, `prefix` is `"geli_da1s3a_keyfile"`.

Not only configure encryption, but also data integrity verification using **HMAC/SHA256**.

```
# geli init -a hmac/sha256 -s 4096 /dev/da0
Enter new passphrase:
Reenter new passphrase:
# geli attach /dev/da0
Enter passphrase:
# dd if=/dev/random of=/dev/da0.eli bs=1m
# newfs /dev/da0.eli
# mount /dev/da0.eli /mnt/secret
```

geli writes the metadata backup by default to the `/var/backups/<prov>.eli` file. If the metadata is lost in any way (e.g., by accidental overwrite), it can be restored. Consider the following situation:

```
# geli init /dev/da0
Enter new passphrase:
Reenter new passphrase:

Metadata backup can be found in /var/backups/da0.eli and
can be restored with the following command:
```

```
# geli restore /var/backups/da0.eli /dev/da0

# geli clear /dev/da0
# geli attach /dev/da0
geli: Cannot read metadata from /dev/da0: Invalid argument.
# geli restore /var/backups/da0.eli /dev/da0
# geli attach /dev/da0
Enter passphrase:
```

If an encrypted filesystem is extended, it is necessary to relocate and update the metadata:

```
# gpart create -s GPT ada0
# gpart add -s 1g -t freebsd-ufs -i 1 ada0
# geli init -K keyfile -P ada0p1
# gpart resize -s 2g -i 1 ada0
# geli resize -s 1g ada0p1
# geli attach -k keyfile -p ada0p1
```

Initialize provider with the passphrase split into two files. The provider can be attached using those two files or by entering "foobar" as the passphrase at the **geli** prompt:

```
# echo foo > da0.pass0
# echo bar > da0.pass1
# geli init -J da0.pass0 -J da0.pass1 da0
# geli attach -j da0.pass0 -j da0.pass1 da0
# geli detach da0
# geli attach da0
Enter passphrase: foobar
```

Suspend all **geli** devices on a laptop, suspend the laptop, then resume devices one by one after resuming the laptop:

```
# geli suspend -a
# zzz
<resume your laptop>
# geli resume -p -k keyfile gpt/secret
# geli resume gpt/private
Enter passphrase:
```

To create a **geli** encrypted filesystem with a file as storage device follow this example. First a file named `private0` is created in `/usr` and attached as a memory disk like `/dev/md0` for example.

```
# dd if=/dev/zero of=/usr/private0 bs=1m count=256
# chmod 0600 /usr/private0
# mdconfig -t vnode -f /usr/private0
```

It is recommended to place the following line in [rc.conf\(5\)](#) to have the memory disk automatically created during boot.

```
mdconfig_md0="-t vnode -f /usr/private0"
```

After `/dev/md0` is created a random key has to be generated and stored in a secure location, like `/root` for example. This key should be protected by a passphrase, which is requested when `geli init` is called.

```
# dd if=/dev/random of=/root/private0.key bs=64 count=1
# geli init -K /root/private0.key -s 4096 /dev/md0
Enter new passphrase:
Reenter new passphrase:
# geli attach -k /root/private0.key /dev/md0
Enter passphrase:
# dd if=/dev/random of=/dev/md0.eli bs=1m
```

Once the initialization of the `/dev/md0.eli` device is ready create a UFS filesystem and mount it for example in `/private`.

```
# newfs /dev/md0.eli
# mount /dev/md0.eli /private
```

After a system reboot the `geli` device can be mounted again with the following commands. The call of `geli attach` will ask for the passphrase. It is recommended to do this procedure after the boot, because otherwise the boot process would be waiting for the passphrase input.

```
# geli attach -k /root/private0.key /dev/md0
Enter passphrase:
# mount /dev/md0.eli /private
```

ENCRYPTION MODES

`geli` supports two encryption modes: **XTS**, which was standardized as **IEEE P1619** and **CBC** with unpredictable IV. The **CBC** mode used by `geli` is very similar to the mode **ESSIV**.

DATA AUTHENTICATION

`geli` can verify data integrity when an authentication algorithm is specified. When data corruption/modification is detected, `geli` will not return any data, but instead will return an error (EINVAL). The offset and size of the corrupted data will be printed on the console. It is important to know against which attacks `geli` provides protection for your data. If data is modified in-place or copied from one place on the disk to another even without modification, `geli` should be able to detect such a change. If an attacker can remember the encrypted data, he can overwrite any future changes with the data he owns without it being noticed. In other words `geli` will not protect your data against replay attacks.

It is recommended to write to the whole provider before first use, in order to make sure that all sectors and their corresponding checksums are properly initialized into a consistent state. One can safely ignore data authentication errors that occur immediately after the first time a provider is attached and before it is initialized in this way.

SEE ALSO

[crypto\(4\)](#), [geom\(4\)](#), [loader.conf\(5\)](#), [geom\(8\)](#), [crypto\(9\)](#)

HISTORY

The `geli` utility appeared in FreeBSD 6.0. Support for the **Camellia** block cipher was implemented by Yoshisato Yanagisawa in FreeBSD 7.0.

Highest **GELI** metadata version supported by the given FreeBSD version:

FreeBSD version	GELI version
6.0	0
6.1	0
6.2	3
6.3	3
6.4	3

7.0	3
7.1	3
7.2	3
7.3	3
7.4	3
8.0	3
8.1	3
8.2	5
9.0	6
10.0	7

AUTHORS

Paweł Jakub Dawidek <pjd@FreeBSD.org>

[NAME](#) | [SYNOPSIS](#) | [DESCRIPTION](#) | [KEY SUMMARY](#) | [SYSCTL VARIABLES](#) | [PERFORMANCE CONSIDERATIONS](#) | [EXIT STATUS](#) | [EXAMPLES](#) | [ENCRYPTION MODES](#) | [DATA AUTHENTICATION](#) | [SEE ALSO](#) | [HISTORY](#) | [AUTHORS](#)

Want to link to this manual page? Use this URL:
<<https://man.freebsd.org/cgi/man.cgi?query=geli&manpath=FreeBSD+15.1-RELEASE+and+Ports.quarterly>>

[home](#) | [help](#)